

Introduction to Programming

Problem Sheet 1: Solutions

No imports should be required

Section 1

1. Identity function on Integers.

```
id' :: Integer -> Integer
id' n = n
```

2. Project first element of a pair of Integers.

```
fst' :: (Integer, Integer) -> Integer
fst' (n, _) = n
```

3. Project second element of a pair of Integers.

```
snd' :: (Integer, Integer) -> Integer
snd' (_, m) = m
```

4. Integer 1-digit adder. Add two single-digit integers (0-9). (Assume both inputs are in the range (0-9).) The first component should have the unit digit of the answer, and the second component should be the carry-over amount

```
addDigit :: Integer -> Integer -> ( Integer , Integer )
addDigit d1 d2 = let x = d1 + d2 in
  if x < 10 then (x, 0) else (x - 10, 1)
```

Section 2

6. Maximum of two Integers.

```
max' :: Integer -> Integer -> Integer
max' n m = if n >= m then n else m
```

7. Minimum of two Integers.

```
min' :: Integer -> Integer -> Integer
min' n m = if n <= m then n else m
```

8. Vector addition: add two 2D vectors component-wise. Vector represented as (x, y).

```
vecAdd :: (Integer, Integer) -> (Integer, Integer) -> (Integer, Integer)
vecAdd (x1, y1) (x2, y2) = (x1 + x2, y1 + y2)
```

9. Scalar multiplication: multiply a 2D vector by a scalar.

```
scalarMul :: Integer -> (Integer, Integer) -> (Integer, Integer)
scalarMul c (x, y) = (c*x, c*y)
```

Section 3

```
toLinearMap :: ((Integer, Integer), (Integer, Integer))
              -> ((Integer, Integer) -> (Integer, Integer))
toLinearMap ((a, b), (c, d)) = t where
    t (x, y) = (a*x + b*y, c*x + d*y)

fromLinearMap :: ((Integer, Integer) -> (Integer, Integer))
               -> ((Integer, Integer), (Integer, Integer))
fromLinearMap t = (
    (a, b),
    (c, d)
  ) where
    (a, c) = t (1, 0)
    (b, d) = t (0, 1)
```

10. Matrix addition (2×2 matrices represented as two rows).

```
matAdd :: ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
matAdd m1 m2 = let t1 = toLinearMap m1 ; t2 = toLinearMap m2 in
    fromLinearMap ( \ v -> t1 v `vecAdd` t2 v )
```

11. Matrix transpose (swap rows and columns).

```
transpose :: ((Integer, Integer), (Integer, Integer))
           -> ((Integer, Integer), (Integer, Integer))
transpose m = fromLinearMap t where
    t v = fst ( matMul (v, v) m )
```

Section 4

12. Matrix multiplication (2×2).

```
matMul :: ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
        -> ((Integer, Integer), (Integer, Integer))
matMul m1 m2 = let t1 = toLinearMap m1 ; t2 = toLinearMap m2 in
    fromLinearMap ( t1 . t2 )
```

Section 5

13. Pair combinator Given two functions $f :: \text{Integer} \rightarrow \text{Integer}$ and $g :: \text{Integer} \rightarrow \text{Integer}$, produce a function that takes an Integer x and returns (f x, g x).

```
pair :: (Integer -> Integer) -> (Integer -> Integer)
      -> ( Integer -> (Integer, Integer) )
pair f g = h where h x = (f x, g x)
```

Section 6

5. Recall the bijection between $A^{(B \times C)}$ and $(A^B)^C$ Given the following function,

```
myFunc :: (Integer, Integer) -> Integer -- A^(B x C) where A = B = C = Integer
myFunc (x,y) = x*x*y + y*(x*x+1) - 120*(y - x*y)
```

there must be a corresponding function

```
corrFunc :: Integer -> ( Integer -> Integer ) -- (A^B)^C where A = B = C = Integer
corrFunc x = g where
  g :: Integer -> Integer
  g y = myFunc (x,y)
```

14. Bijection between Integer and (Integer, Integer) Define a bijection (one-to-one and onto) between integers and pairs of integers. That is, give functions toPair :: Integer -> (Integer, Integer) and fromPair :: (Integer, Integer) -> Integer such that they are inverses.

<https://q.uiver.app/#q=WzAsMTYsWzAsNCwiMjdlFsdUsIjEiXSxbMCwzLCI2Il0sWzIsNSwiNCJdLFsx>

```
fromPair :: (Integer, Integer) -> Integer
fromPair (x,y) = if x > 0 || ( x == 0 && y >= 0 )
  then (x + abs y)^2 + y
  else - fromPair (-x,-y)

toPair :: Integer -> (Integer, Integer)
toPair z = case signum z of
  -1 -> (-x,-y) where (x,y) = toPair (-z)
  0 -> (0,0)
  1 -> let (x, y) = toPair (z - 1) in
    if x == 0 then (1, -y) else (x + if y < 0 then 1 else -1, y + 1)
```